

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA | INGENIERÍA INFORMÁTICA
LENGUAJE Y COMPILADORES - SECCIÓN “1”

PRUEBA DIAGNOSTICA

Grupo LC

Eslogan: Pensamiento, Analisis y Ejecucion.

Autora: V-28.550.912 | Sanchez, Rolannys

Ejercicio 1. Validación de movimientos de caballo en un tablero de ajedrez

Descripción de la solución

Para desarrollar este ejercicio se implementó un programa en Python que permite verificar una secuencia de movimientos realizados por un caballo sobre un tablero de ajedrez de 8 × 8 casillas. El programa recibe una posición inicial y una secuencia de movimientos almacenados en un archivo de texto denominado secuencia.txt.

La solución comprueba que cada movimiento tenga el formato establecido, que la casilla de destino se encuentre dentro de los límites del tablero y que el desplazamiento corresponda al movimiento permitido para una pieza de tipo caballo. En caso de encontrar un movimiento incorrecto, el proceso de validación se detiene e indica el número del primer movimiento que presenta el error.

Además, se incorporó una medición del tiempo de ejecución para realizar diez validaciones de la secuencia y calcular el tiempo acumulado y el promedio obtenido.

Funciones principales

Para organizar el desarrollo se dividió el programa en funciones, cada una encargada de una tarea específica:

Función convertir_posicion(casilla)

Se encarga de convertir una posición del tablero, representada mediante una letra y un número (por ejemplo, b4), en coordenadas numéricas. También verifica que la columna se encuentre entre las letras a y h, y que la fila esté comprendida entre los números 1 y 8. De esta manera, se comprueba que la posición pertenezca al tablero.

```python
# Convierte una casilla en coordenadas del tablero.
def convertir_posicion(casilla):

    if len(casilla) != 2:
        return None

    columna = casilla[0]
    fila = casilla[1]

    if columna < 'a' or columna > 'h' or fila < '1' or fila > '8':
        return None

    x = ord(columna) - ord('a')
    y = int(fila) - 1

    return (x, y)

```

Función movimiento_valido(origen, destino)

Comprueba si el desplazamiento realizado por el caballo cumple con las reglas de movimiento de esta pieza. Para ello, calcula la diferencia absoluta entre las coordenadas de origen y destino, verificando que el movimiento corresponda a dos casillas en una dirección y una en la otra, formando una L.

```python
# Comprueba si el caballo realiza un movimiento valido.
def movimiento_valido(origen, destino):

    dx = abs(destino[0] - origen[0])
    dy = abs(destino[1] - origen[1])

    return (dx == 2 and dy == 1) or (dx == 1 and dy == 2)
```

Función validar_secuencia(posicion_inicial, movimientos)

Es la función principal de validación. Recibe la posición inicial y la lista de movimientos, comprueba que cada uno tenga el formato correcto, verifica que las casillas estén dentro del tablero y determina si el desplazamiento es legal. Cuando encuentra el primer movimiento incorrecto, interrumpe el recorrido y devuelve su número junto con el motivo del error.

```python
# Valida todos los movimientos de la secuencia.
def validar_secuencia(posicion_inicial, movimientos):

    actual = convertir_posicion(posicion_inicial)

    # Comprobar la posicion inicial.
    if actual is None:
        return False, 0, "Posicion inicial invalida"

    # Recorrer todos los movimientos.
    for i, movimiento in enumerate(movimientos):

        # Comprobar el formato del movimiento.
        if len(movimiento) != 3 or movimiento[0] != 'K':
            return False, i + 1, "Formato de movimiento invalido"

        # Obtener la casilla de destino.
        casilla = movimiento[1:3]

        siguiente = convertir_posicion(casilla)

        # Comprobar que la casilla este dentro del tablero.
        if siguiente is None:
            return False, i + 1, "Casilla fuera del tablero"

        # Comprobar que el movimiento sea legal.
        if not movimiento_valido(actual, siguiente):
            return False, i + 1, "Movimiento ilegal del caballo"

        # Actualizar la posicion actual.
        actual = siguiente

    return True, 0, "Todos los movimientos son validos"
```

Función main()

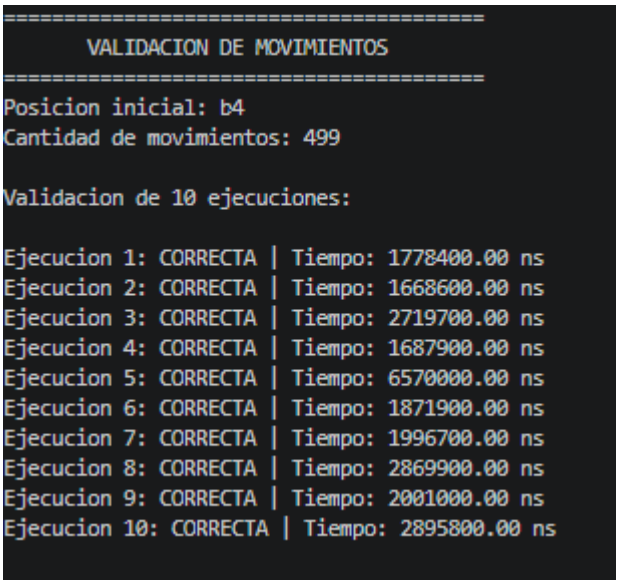
Coordina la ejecución del programa. Localiza y lee el archivo secuencia.txt, obtiene la posición inicial y los movimientos restantes, ejecuta diez validaciones y registra el tiempo empleado en cada una. Finalmente, presenta los resultados y calcula el tiempo acumulado y el promedio de ejecución.

Pruebas y resultados

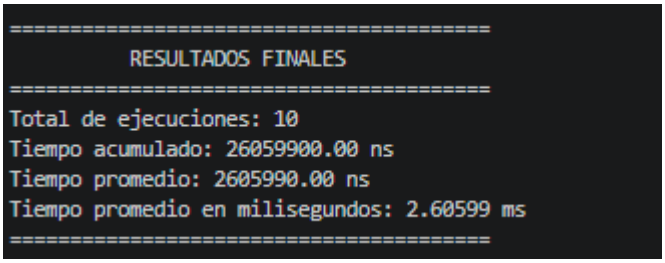
Para comprobar el funcionamiento del programa se utilizó el archivo secuencia.txt, que contiene la posición inicial del caballo y la secuencia de movimientos que debe validar. En esta prueba se estableció como posición inicial la casilla b4 y se procesaron 499 movimientos.

El programa realizó diez ejecuciones consecutivas. En todas ellas, la secuencia fue validada correctamente, sin detectar movimientos ilegales ni errores de formato. Esto permitió comprobar que el procedimiento verifica los desplazamientos del caballo y recorre satisfactoriamente la secuencia proporcionada.

Los tiempos registrados durante las ejecuciones fueron los siguientes:



Resumen de los resultados:



Los resultados muestran que el programa logró validar satisfactoriamente todos los movimientos de la secuencia en las diez ejecuciones. El tiempo promedio obtenido fue de aproximadamente 2,60 milisegundos, lo que representa el tiempo medio empleado para recorrer y comprobar los 499 movimientos en cada ejecución.

Ejercicio 2. Generación de coeficientes del polinomio mediante el triángulo de Pascal

Descripción de la solución

Para desarrollar este ejercicio se implementaron dos programas, uno en lenguaje C y otro en Python, con la finalidad de generar los coeficientes del desarrollo del binomio ((x+1)^n) mediante el triángulo de Pascal.

Ambas implementaciones solicitan al usuario un valor entero no negativo (n) y un valor para (x). A partir de estos datos, generan los coeficientes correspondientes, construyen y muestran el polinomio resultante y realizan su evaluación, presentando el cálculo de cada término.

En el caso de C, se emplea memoria dinámica para almacenar los coeficientes, mientras que en Python se utilizan listas. Además, ambos programas realizan diez mediciones del tiempo necesario para generar los coeficientes, con el propósito de comparar el rendimiento de las dos implementaciones.

Desarrollo e implementación

Para resolver este ejercicio se desarrollaron dos implementaciones, una en lenguaje C y otra en Python.

El procedimiento se basa en las propiedades del triángulo de Pascal. Cada fila representa los coeficientes de una potencia del binomio. Los valores de los extremos siempre son uno, mientras que los elementos internos se obtienen sumando los dos valores que se encuentran encima de ellos.

De esta manera, se construyen progresivamente las filas hasta alcanzar el valor de (n) solicitado.

En la implementación en C se utilizó memoria dinámica para almacenar los coeficientes, permitiendo reservar y administrar el espacio requerido durante la ejecución. En Python se utilizaron estructuras dinámicas para realizar el mismo procedimiento.

Una vez generados los coeficientes, el programa construye la representación del polinomio y realiza su evaluación para el valor de (x) introducido.

Funciones principales de la implementación en C

El programa desarrollado en C se organizó mediante funciones que permiten separar la generación de coeficientes, su presentación, la evaluación del polinomio y la medición del tiempo.

Función copiar_cadena()

Reserva memoria dinámica para almacenar una copia de una cadena de caracteres. Se utiliza para administrar los coeficientes, que son representados como cadenas de texto.

```c
/* Reserva memoria y copia una cadena */
char *copiar_cadena(const char *texto) {
    char *copia = (char *)malloc(strlen(texto) + 1);

    if (copia == NULL) {
        printf("Error al reservar memoria.\n");
        exit(1);
    }

    strcpy(copia, texto);
    return copia;
}
```

Función sumar()

Realiza la suma de dos números enteros grandes representados como cadenas de caracteres. Opera dígito por dígito y contempla el acarreo, permitiendo obtener resultados que pueden superar la capacidad de los tipos enteros convencionales.

```c
/* Suma dos numeros enteros grandes representados como cadenas */
char *sumar(const char *a, const char *b) {
    int longitud_a = (int)strlen(a);
    int longitud_b = (int)strlen(b);

    int maximo = longitud_a > longitud_b ? longitud_a : longitud_b;

    char *resultado = (char *)malloc(maximo + 2);

    if (resultado == NULL) {
        printf("Error al reservar memoria.\n");
        exit(1);
    }

    int i = longitud_a - 1;
    int j = longitud_b - 1;
    int k = maximo;
    int acarreo = 0;

    resultado[maximo + 1] = '\0';

    while (i >= 0 || j >= 0 || acarreo > 0) {
        int digito_a = (i >= 0) ? a[i] - '0' : 0;
        int digito_b = (j >= 0) ? b[j] - '0' : 0;

        int suma = digito_a + digito_b + acarreo;

        resultado[k] = (char)('0' + (suma % 10));
        acarreo = suma / 10;

        i--;
        j--;
        k--;
    }

    int inicio = k + 1;

    memmove(resultado, resultado + inicio,
            (size_t)(maximo + 2 - inicio));

    return resultado;
}
```

Función generar_pascal()

Genera los coeficientes del triángulo de Pascal hasta la fila indicada por (n). Para ello, reserva dinámicamente un arreglo de cadenas y actualiza sus valores mediante sumas sucesivas. Esta función constituye el procedimiento principal para obtener los coeficientes del polinomio.

```c
/* Genera los coeficientes mediante el triangulo de Pascal */
char **generar_pascal(int n) {
    char **fila = (char **)malloc((n + 1) * sizeof(char *));

    if (fila == NULL) {
        printf("Error al reservar memoria.\n");
        exit(1);
    }

    for (int i = 0; i <= n; i++) {
        fila[i] = copiar_cadena("0");
    }

    free(fila[0]);
    fila[0] = copiar_cadena("1");

    for (int i = 1; i <= n; i++) {
        for (int j = i; j >= 1; j--) {
            char *nuevo = sumar(fila[j], fila[j - 1]);

            free(fila[j]);
            fila[j] = nuevo;
        }
    }

    return fila;
}
```

Función liberar_pascal ()

Libera la memoria dinámica utilizada durante la generación de los coeficientes. Recorre las cadenas almacenadas, libera cada una y finalmente libera el arreglo principal.

```c
/* Libera la memoria dinamica utilizada */
void liberar_pascal(char **fila, int n) {
    if (fila == NULL) {
        return;
    }

    for (int i = 0; i <= n; i++) {
        free(fila[i]);
    }

    free(fila);
}
```

Función mostrar_polinomio()

Utiliza los coeficientes generados para construir y presentar el polinomio desarrollado, organizando su impresión en bloques de cinco términos por línea para facilitar su lectura.

```c
/* Muestra el polinomio organizado en cinco terminos por linea */
void mostrar_polinomio(char **fila, int n) {
    printf("\nPOLINOMIO DESARROLLADO:\n\n");

    for (int i = 0; i <= n; i++) {
        int exponente = n - i;

        printf("%s", fila[i]);

        if (exponente > 1) {
            printf("x^%d", exponente);
        } else if (exponente == 1) {
            printf("x");
        }

        if (i < n) {
            printf(" + ");
        }

        if ((i + 1) % TERMINOS_POR_LINEA == 0 && i < n) {
            printf("\n");
        }
    }

    printf("\n");
}
```

Función evaluar_polinomio()

Evalúa el polinomio utilizando el valor de (x) introducido por el usuario. Calcula cada término mediante su coeficiente y la potencia correspondiente, muestra las operaciones realizadas y acumula los resultados para obtener el valor final. También realiza una comprobación directa utilizando la expresión ((x+1)^n).

```c
/* Evalua el polinomio mostrando el calculo de cada termino */
double evaluar_polinomio(char **fila, int n, double x) {
    double resultado = 0.0;

    printf("\nEVALUACION DEL POLINOMIO:\n\n");

    for (int i = 0; i <= n; i++) {
        int exponente = n - i;

        double coeficiente = strtod(fila[i], NULL);
        double potencia = pow(x, exponente);
        double termino = coeficiente * potencia;

        printf("Termino %d: %s * %.10g^%d = %.10g\n",
               i + 1, fila[i], x, exponente, termino);

        resultado += termino;
    }

    printf("\nResultado de la suma de los terminos: %.10e\n",
           resultado);

    printf("Comprobacion directa (x+1)^n: %.10e\n",
           pow(x + 1.0, n));

    return resultado;
}
```

Función medir_tiempo()

Mide el tiempo empleado exclusivamente en generar los coeficientes mediante generar_pascal(). Para realizar esta medición utiliza las funciones QueryPerformanceCounter() y QueryPerformanceFrequency() de Windows, que

permiten obtener intervalos de tiempo de alta precisión. Después de la medición, libera la memoria utilizada.

```c
/* Mide el tiempo de generacion de los coeficientes */
double medir_tiempo(int n) {
    LARGE_INTEGER frecuencia;
    LARGE_INTEGER inicio;
    LARGE_INTEGER fin;

    QueryPerformanceFrequency(&frecuencia);

    QueryPerformanceCounter(&inicio);

    char **temporal = generar_pascal(n);

    QueryPerformanceCounter(&fin);

    liberar_pascal(temporal, n);

    return ((double)(fin.QuadPart - inicio.QuadPart) /
            frecuencia.QuadPart) * 1000.0;
}
```

Función comparar_tiempos()

Ejecuta diez mediciones de la generación de coeficientes, almacena los tiempos obtenidos y calcula el promedio. También guarda los resultados en el archivo resultados_c.txt.

```c
/* Ejecuta diez mediciones y calcula el promedio */
void comparar_tiempos(int n) {
    double tiempos[REPETICIONES];
    double suma = 0.0;
    double promedio;

    printf("\nMEDICION DEL TIEMPO DE EJECUCION EN C:\n\n");

    for (int i = 0; i < REPETICIONES; i++) {
        tiempos[i] = medir_tiempo(n);
        suma += tiempos[i];

        printf("Ejecucion %d: %.8f ms\n",
               i + 1, tiempos[i]);
    }

    promedio = suma / REPETICIONES;

    printf("\nPromedio de las %d ejecuciones: %.8f ms\n",
           REPETICIONES, promedio);

    /* Guarda los resultados en un archivo TXT */
    FILE *archivo = fopen("resultados_c.txt", "w");

    if (archivo != NULL) {
        fprintf(archivo, "RESULTADOS DE EJECUCION EN C\n");
        fprintf(archivo, "Valor de n: %d\n", n);
        fprintf(archivo, "Repeticiones: %d\n\n", REPETICIONES);

        for (int i = 0; i < REPETICIONES; i++) {
            fprintf(archivo, "Ejecucion %d: %.8f ms\n",
                    i + 1, tiempos[i]);
        }

        fprintf(archivo, "\nPromedio: %.8f ms\n", promedio);

        fclose(archivo);

        printf("\nResultados guardados en resultados_c.txt\n");
    } else {
        printf("\nNo se pudo crear el archivo de resultados.\n");
    }
}
```

Funciones principales de la implementación en Python

La versión desarrollada en Python mantiene el mismo procedimiento general, utilizando las estructuras dinámicas propias del lenguaje.

Función generar_pascal(n)

Crea una lista con (n+1) posiciones para almacenar los coeficientes. Inicializa el primer elemento con uno y genera las filas siguientes mediante sumas sucesivas, recorriendo las posiciones de derecha a izquierda para conservar los valores de la fila anterior durante cada actualización.

```python
def generar_pascal(n):
    """Genera la fila n del triangulo de Pascal."""
    coeficientes = [0] * (n + 1)
    coeficientes[0] = 1

    for fila in range(1, n + 1):
        coeficientes[fila] = 1

        for columna in range(fila - 1, 0, -1):
            coeficientes[columna] += coeficientes[columna - 1]

    return coeficientes
```

Función mostrar_polinomio(coeficientes, n)

Construye la expresión algebraica utilizando los coeficientes generados y sus respectivos exponentes. Organiza la salida en bloques de cinco términos por línea para facilitar su visualización.

```python
def mostrar_polinomio(coeficientes, n):
    """Muestra el polinomio generado en bloques de cinco terminos."""
    print("\nPolinomio generado:")

    contador = 0

    for exp in range(n, -1, -1):
        coef = coeficientes[exp]

        if exp == 0:
            termino = str(coef)
        elif exp == 1:
            termino = "x" if coef == 1 else f"{coef}x"
        else:
            termino = f"x^{exp}" if coef == 1 else f"{coef}x^{exp}"

        if contador > 0:
            print(" + ", end="")

        print(termino, end="")

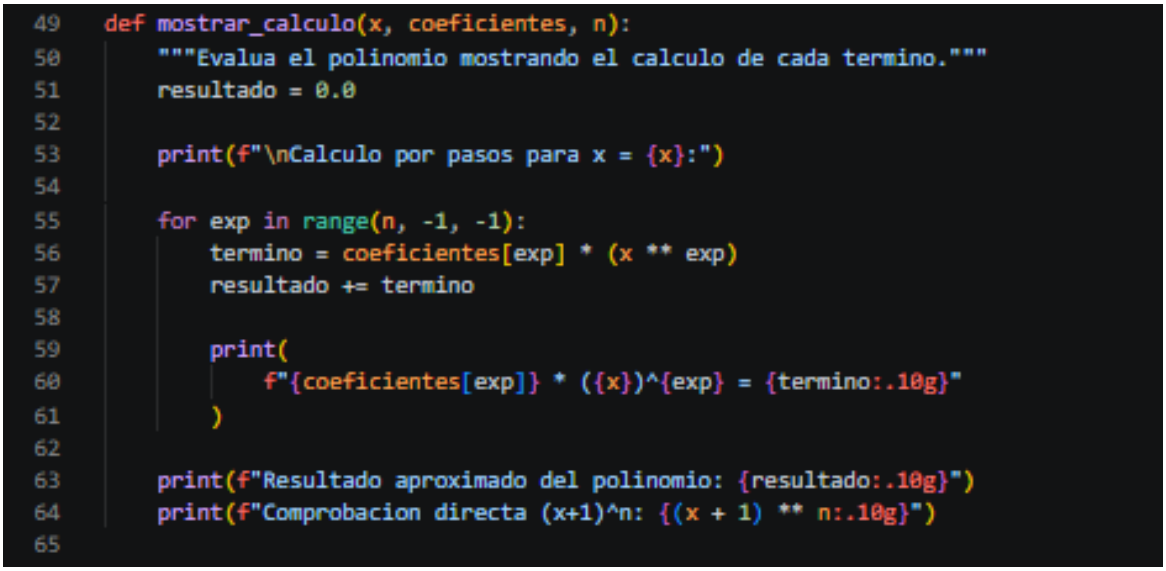
        contador += 1

        if contador == 5 and exp != 0:
            print()
            contador = 0

    print()
```

Función mostrar_calculo(x, coeficientes, n)

Realiza la evaluación del polinomio término por término. Para cada exponente calcula el producto entre el coeficiente y la potencia correspondiente de (x), muestra la operación y acumula el resultado. Finalmente, presenta el resultado aproximado y una comprobación directa mediante ((x+1)^n).



```
49  def mostrar_calculo(x, coeficientes, n):
50      """Evalua el polinomio mostrando el calculo de cada termino."""
51      resultado = 0.0
52
53      print(f"\nCalculo por pasos para x = {x}:")
54
55      for exp in range(n, -1, -1):
56          termino = coeficientes[exp] * (x ** exp)
57          resultado += termino
58
59          print(
60              f"{coeficientes[exp]} * ({x})^{exp} = {termino:.10g}"
61          )
62
63      print(f"Resultado aproximado del polinomio: {resultado:.10g}")
64      print(f"Comprobacion directa (x+1)^n: {(x + 1) ** n:.10g}")
65
```

Función comparar_tiempos(n)

Realiza diez mediciones del tiempo empleado por generar_pascal(n), utilizando time.perf_counter() para registrar los intervalos. Posteriormente calcula el tiempo total y el promedio, muestra los resultados y los almacena en el archivo resultados_python.txt.

```python
def comparar_tiempos(n):
    """Mide diez generaciones de la fila n del triangulo de Pascal."""
    repeticiones = 10
    tiempos = []

    for _ in range(repeticiones):
        inicio = time.perf_counter()

        temporal = generar_pascal(n)

        fin = time.perf_counter()

        tiempos.append((fin - inicio) * 1000)

    total = sum(tiempos)
    promedio = total / repeticiones

    with open("resultados_python.txt", "w", encoding="utf-8") as archivo:
        archivo.write("Ejercicio 2 - Triangulo de Pascal (Python)\n")
        archivo.write(f"n = {n}\n")
        archivo.write(f"Repeticiones = {repeticiones}\n\n")

        archivo.write("Tiempos de ejecucion:\n")

        for i, tiempo in enumerate(tiempos, start=1):
            archivo.write(f"Ejecucion {i}: {tiempo:.8f} ms\n")

        archivo.write(f"\nTiempo total: {total:.8f} ms\n")
        archivo.write(f"Tiempo promedio: {promedio:.8f} ms\n")

    print("\nTiempos de las 10 generaciones:")

    for i, tiempo in enumerate(tiempos, start=1):
        print(f"Ejecucion {i}: {tiempo:.8f} ms")

    print(f"Tiempo total: {total:.8f} ms")
    print(f"Tiempo promedio: {promedio:.8f} ms")
    print("Resultados guardados en resultados_python.txt")


```

Uso de memoria dinámica en C

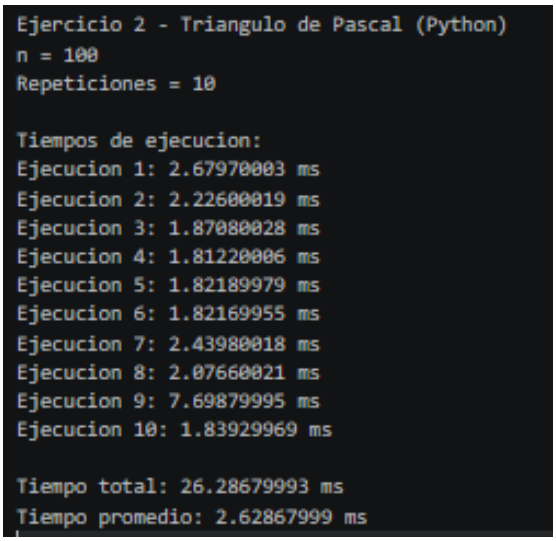
Para cumplir con el requerimiento de memoria dinámica, la implementación en C utiliza las funciones malloc() y free() para reservar y liberar el espacio necesario durante la ejecución.

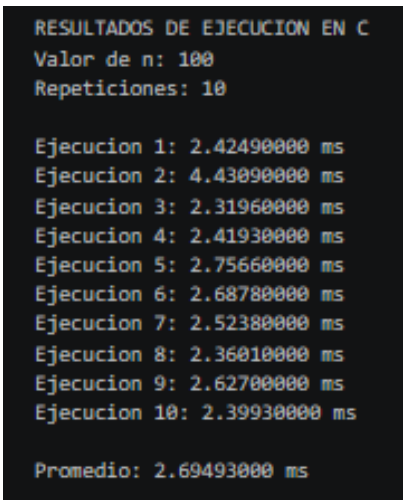
Los coeficientes se almacenan en un arreglo dinámico de cadenas de caracteres (**char**). Esta representación permite manejar números de diferentes longitudes sin depender de un tamaño fijo para cada coeficiente.

Además, la función sumar() reserva memoria para almacenar el resultado de cada operación. Cuando se actualiza un coeficiente, se libera la memoria de la cadena anterior y se sustituye por la nueva. Al finalizar el procesamiento, la función liberar_pascal() libera los recursos utilizados.
En Python, el almacenamiento se realiza mediante listas, cuyo tamaño se determina a partir del valor de (n) y cuyos elementos pueden almacenar enteros de precisión arbitraria.

Comparación de tiempos de ejecución

Para comparar el rendimiento de ambas implementaciones se utilizó el valor (n=100), con el cual se generan 101 coeficientes correspondientes al desarrollo del binomio.
Se realizaron diez mediciones en cada lenguaje, considerando exclusivamente el tiempo necesario para generar los coeficientes mediante el triángulo de Pascal.
Los resultados obtenidos fueron los siguientes:





De acuerdo con los resultados, Python presentó un menor tiempo promedio de ejecución, con una diferencia aproximada de 0,06625 milisegundos respecto a C. Aunque la diferencia entre ambos resultados es pequeña, las mediciones muestran que Python obtuvo un tiempo promedio ligeramente inferior en esta

prueba específica. Es importante considerar que los tiempos pueden variar entre ejecuciones debido a las condiciones del equipo y a los procesos que se encuentran activos durante la medición.

Por lo tanto, esta comparación permite observar el comportamiento de las dos implementaciones bajo las mismas condiciones de entrada, sin establecer que uno de los lenguajes sea necesariamente más rápido en todos los casos.

Ejercicio 3. Traducción de palabras reservadas del lenguaje C

Funciones principales

traducir_palabra()

Se encarga de buscar cada identificador dentro de un diccionario que relaciona las palabras reservadas de C11 con sus equivalentes en español. La comparación se realiza mediante strcmp(), por lo que solamente se traducen las coincidencias exactas.

```c
/* Comprueba si un identificador es una palabra reservada */
const char *traducir_palabra(const char *palabra) {

    for (int i = 0; i < TOTAL_RESERVADAS; i++) {

        if (strcmp(palabra, reservadas[i].ingles) == 0) {
            return reservadas[i].espanol;
        }
    }

    return NULL;
}
```

cargar_archivo()

Abre el archivo fuente y carga su contenido en memoria dinámica mediante malloc(). Cuando el espacio reservado resulta insuficiente, utiliza realloc() para ampliar la capacidad. De esta manera, el programa puede procesar archivos sin depender de un tamaño fijo previamente establecido.



```c
/* Carga el archivo utilizando memoria dinamica */
char *cargar_archivo(const char *nombre, size_t *longitud) {

    FILE *archivo = fopen(nombre, "rb");

    if (archivo == NULL) {
        printf("Error: no se pudo abrir el archivo %s\n", nombre);
        return NULL;
    }
    size_t capacidad = 1024;
    size_t usados = 0;

    char *contenido = malloc(capacidad);

    if (contenido == NULL) {
        printf("Error al reservar memoria.\n");
        fclose(archivo);
        return NULL;
    }
    int caracter;

    while ((caracter = fgetc(archivo)) != EOF) {

        if (usados + 1 >= capacidad) {

            capacidad *= 2;

            char *temporal = realloc(contenido, capacidad);

            if (temporal == NULL) {
                printf("Error al ampliar la memoria.\n");
                free(contenido);
                fclose(archivo);
                return NULL;
            }

            contenido = temporal;
        }

        contenido[usados++] = (char)caracter;
    }
    contenido[usados] = '\0';
    *longitud = usados;

    fclose(archivo);

    return contenido;
}
```

agregar_texto()

Incorpora fragmentos de texto al resultado de la traducción. Comprueba si existe suficiente espacio disponible y, cuando es necesario, amplía la memoria utilizando realloc(). Esta función permite construir progresivamente el código traducido.



```c
/* Agrega texto al resultado y amplia su memoria cuando es necesario */
int agregar_texto(char **resultado, size_t *usados,
                  size_t *capacidad, const char *texto) {

    size_t longitud = strlen(texto);

    if (*usados + longitud + 1 > *capacidad) {

        size_t nueva_capacidad = *capacidad;

        while (*usados + longitud + 1 > nueva_capacidad) {
            nueva_capacidad *= 2;
        }

        char *temporal = realloc(*resultado, nueva_capacidad);

        if (temporal == NULL) {
            return 0;
        }

        *resultado = temporal;
        *capacidad = nueva_capacidad;
    }

    memcpy(*resultado + *usados, texto, longitud);

    *usados += longitud;
    (*resultado)[*usados] = '\0';

    return 1;
}
```

analizar_codigo()

Es la función principal del proceso de traducción. Recorre el contenido del archivo carácter por carácter y reconoce diferentes elementos del código:

- **Comentarios:** identifica los comentarios de una línea y de varias líneas, conservándolos sin modificaciones.
- **Cadenas de texto y caracteres:** reconoce las comillas y mantiene su contenido original, incluyendo los caracteres de escape.
- **Identificadores:** reconoce palabras completas y comprueba si pertenecen al diccionario de palabras reservadas. Si encuentra una coincidencia, agrega su traducción; de lo contrario, conserva el identificador original.
- **Otros caracteres:** copia los espacios, operadores, signos y demás elementos sin modificarlos.

Además, reserva memoria dinámica para cada identificador según su longitud real, evitando depender de un tamaño fijo para almacenar las palabras encontradas.



```c
/* Analiza el codigo y sustituye las palabras reservadas */
char *analizar_codigo(const char *codigo, size_t longitud) {

    size_t capacidad = longitud + 1024;
    size_t usados = 0;
    size_t i = 0;

    char *resultado = malloc(capacidad);

    if (resultado == NULL) {
        return NULL;
    }

    resultado[0] = '\0';

    while (i < longitud) {

        /* Reconocer comentarios de una sola linea */
        if (codigo[i] == '/' && i + 1 < longitud &&
            codigo[i + 1] == '/') {

            while (i < longitud && codigo[i] != '\n') {

                char temporal[2] = {codigo[i], '\0'};

                if (!agregar_texto(&resultado, &usados,
                                   &capacidad, temporal)) {
                    free(resultado);
                    return NULL;
                }

                i++;
            }
        }

        /* Reconocer comentarios de varias lineas */
        else if (codigo[i] == '/' && i + 1 < longitud &&
                 codigo[i + 1] == '*') {

            while (i < longitud) {

                char temporal[2] = {codigo[i], '\0'};

                if (!agregar_texto(&resultado, &usados,
                                   &capacidad, temporal)) {
                    free(resultado);
                    return NULL;
                }

                if (codigo[i] == '*' && i + 1 < longitud &&
                    codigo[i + 1] == '/') {

                    i++;

                    char cierre[2] = {codigo[i], '\0'};
```

guardar_resultado()

Crea el archivo prueba_traducida.txt y almacena en él el código resultante, permitiendo conservar la traducción realizada.



```c
/* Guarda el resultado en un archivo nuevo */
int guardar_resultado(const char *nombre, const char *contenido) {

    FILE *archivo = fopen(nombre, "w");

    if (archivo == NULL) {
        printf("Error al crear el archivo de salida.\n");
        return 0;
    }

    fputs(contenido, archivo);

    fclose(archivo);

    return 1;
}
```

main()

Coordina la ejecución del programa. Solicita el nombre del archivo fuente, carga su contenido, ejecuta el análisis, muestra las palabras reservadas encontradas y presenta el código traducido. Finalmente, guarda el resultado y libera la memoria dinámica utilizada mediante free().

Aplicación de memoria dinámica

La memoria dinámica constituye una parte fundamental de esta implementación. Se utiliza malloc() para reservar espacio inicialmente, realloc() para ampliar la capacidad cuando el contenido lo requiere y free() para liberar los espacios utilizados al finalizar el procesamiento.

Este mecanismo permite adaptar el almacenamiento al tamaño real del archivo y del resultado generado, evitando establecer una capacidad fija para todo el código fuente.

Pruebas de funcionamiento

Para comprobar el funcionamiento del traductor se realizaron cinco pruebas con diferentes fragmentos de código fuente en C. Estas pruebas permitieron verificar la identificación de palabras reservadas, la conservación de elementos que no deben modificarse y el reconocimiento correcto de identificadores.

Prueba 1. Identificación y traducción de palabras reservadas

Se utilizó un fragmento de código que contenía diferentes palabras reservadas de C, como int, if, return y else.

```
==================================
   TRADUCTOR DE PALABRAS DE C
==================================
Ingrese el nombre del archivo: prueba1.txt

Archivo cargado correctamente.
Tamano del archivo: 125 bytes

PALABRAS RESERVADAS ENCONTRADAS:

Encontrada: int              -> entero
Encontrada: float            -> flotante
Encontrada: char             -> caracter
Encontrada: if               -> si
Encontrada: return           -> retornar
Encontrada: else             -> sino
Encontrada: return           -> retornar

==================================
       CODIGO TRADUCIDO
==================================


entero edad = 25;
flotante altura = 1.75;
caracter inicial = 'R';

si (edad > 18) {
    retornar 1;
} sino {
    retornar 0;
}


Resultado guardado en prueba_traducida.txt

Memoria liberada correctamente.
```

Resultado obtenido: satisfactorio. Las palabras reservadas fueron identificadas y traducidas correctamente.

Prueba 2. Conservación de comentarios

Se introdujeron comentarios de una sola línea (//) y comentarios de varias líneas (/* */) que contenían palabras reservadas.

Resultado esperado: el programa debía conservar los comentarios sin traducir las palabras que aparecieran dentro de ellos.

```
   TRADUCTOR DE PALABRAS DE C
==================================
Ingrese el nombre del archivo: prueba2.txt

Archivo cargado correctamente.
Tamano del archivo: 170 bytes

PALABRAS RESERVADAS ENCONTRADAS:

Encontrada: int              -> entero
Encontrada: return           -> retornar

==================================
       CODIGO TRADUCIDO
==================================


entero edad = 20;

// int, float, char, if, else y return son palabras reservadas.

/*
    Este comentario contiene:
    int
    if
    while
*/

retornar edad;


Resultado guardado en prueba_traducida.txt

Memoria liberada correctamente.
```

Resultado obtenido: satisfactorio. Los comentarios se mantuvieron sin modificaciones.

Prueba 3. Conservación de cadenas de texto y caracteres

Se utilizó código que contenía cadenas entre comillas dobles y caracteres entre comillas simples, incluyendo palabras reservadas dentro de estos elementos.

Resultado esperado: el programa debía mantener el contenido de las cadenas y los caracteres originales, sin realizar traducciones dentro de ellos.

```
==================================
   TRADUCTOR DE PALABRAS DE C
==================================
Ingrese el nombre del archivo: prueba3.txt

Archivo cargado correctamente.
Tamano del archivo: 133 bytes

PALABRAS RESERVADAS ENCONTRADAS:

Encontrada: int              -> entero
Encontrada: char             -> caracter
Encontrada: return           -> retornar

==================================
       CODIGO TRADUCIDO
==================================

#include <stdio.h>

entero main() {
    printf("int float char if else return while");

    caracter inicial = 'A';

    retornar 0;
}

Resultado guardado en prueba_traducida.txt

Memoria liberada correctamente.
```

Resultado obtenido: satisfactorio. Las cadenas de texto y los caracteres se conservaron correctamente.

Prueba 4. Identificación de palabras similares

Se probaron identificadores como integer, entero, floatValue, whileLoop y returnValue, los cuales contienen secuencias de caracteres similares a palabras reservadas, pero no son palabras reservadas de C.

Resultado esperado: el programa debía reconocer únicamente las coincidencias exactas, evitando traducir parcialmente los identificadores.

```
   TRADUCTOR DE PALABRAS DE C
==================================
Ingrese el nombre del archivo: prueba4.txt

Archivo cargado correctamente.
Tamano del archivo: 125 bytes

PALABRAS RESERVADAS ENCONTRADAS:

Encontrada: int              -> entero
Encontrada: int              -> entero
Encontrada: float            -> flotante
Encontrada: int              -> entero
Encontrada: int              -> entero
Encontrada: return           -> retornar

==================================
       CODIGO TRADUCIDO
==================================

entero integer = 10;
entero entero = 20;

flotante floatValue = 5.5;

entero whileLoop = 3;
entero returnValue = 0;

retornar integer;

Resultado guardado en prueba_traducida.txt
```

Resultado obtenido: satisfactorio. Los identificadores se conservaron sin modificaciones.

Prueba 5. Identificadores de gran longitud

Se introdujo un identificador con una longitud superior a la que podría manejar un arreglo de caracteres de tamaño fijo utilizado en una implementación limitada.

Resultado esperado: el programa debía reconocer y conservar el identificador completo, sin truncarlo ni alterar su contenido.



Resultado obtenido: la primera versión presentó una limitación al trabajar con identificadores largos. Para solucionarla, se modificó el procedimiento de reconocimiento para reservar memoria dinámica según la longitud real de cada identificador, utilizando malloc() y copiando su contenido mediante memcpy(). La versión corregida debe conservar el identificador completo.

Análisis de las pruebas

Las pruebas permitieron comprobar que el programa no se limita a buscar palabras reservadas de manera aislada, sino que analiza el contexto en el que aparecen dentro del código fuente. Esto evita modificar comentarios, cadenas de texto, caracteres e identificadores que solamente contienen partes de palabras reservadas, manejo de identificadores de longitud variable mediante memoria dinámica.